

Chapitre 8

Introduction À GitLab CI/CD

Définitions

CI vs CD vs CD

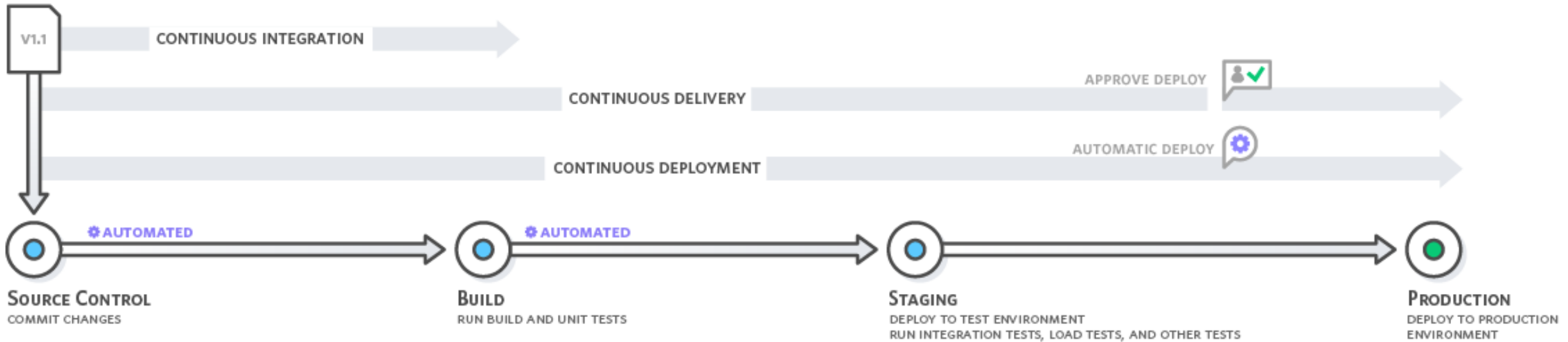


Image : aws.amazon.com

Plan

1. Qu'est-ce que l'intégration ?
2. Quels sont les risques ?
3. La solution : intégration continue
4. Utilisation de Gitlab CI/CD

Qu'est-ce que l'intégration ?

Fusionner **plusieurs sous-systèmes** en un seul.

Chaque sous-parties peut

être développée séparément

utiliser des langages de programmation différents

utiliser des architectures différentes

Projet en C++ : quelles étapes ?

Lister les étapes depuis le développement jusqu'au déploiement

Projet en C++ : quelles étapes ?

LOCAL

Code, compile, link, manual tests, unit tests, integration tests, functional tests, commit, rebase-merge, push

REMOTE

Pull, recompile, all tests, installers, sign binaries, install on VMs, all tests, upload installers.

Quels sont les risques ?

En fonction des technologies utilisées ?

En fonction du temps entre chaque intégration ?

➔ **Integration Hell**

La solution : intégration continue

Intégrer "en continue" → plusieurs fois par jour

CI/CD Permet de :

- trouver les bugs le plus tôt possible

- s'assurer que le produit reste livrable

- vérifier la portabilité (Windows / Mac / Linux)

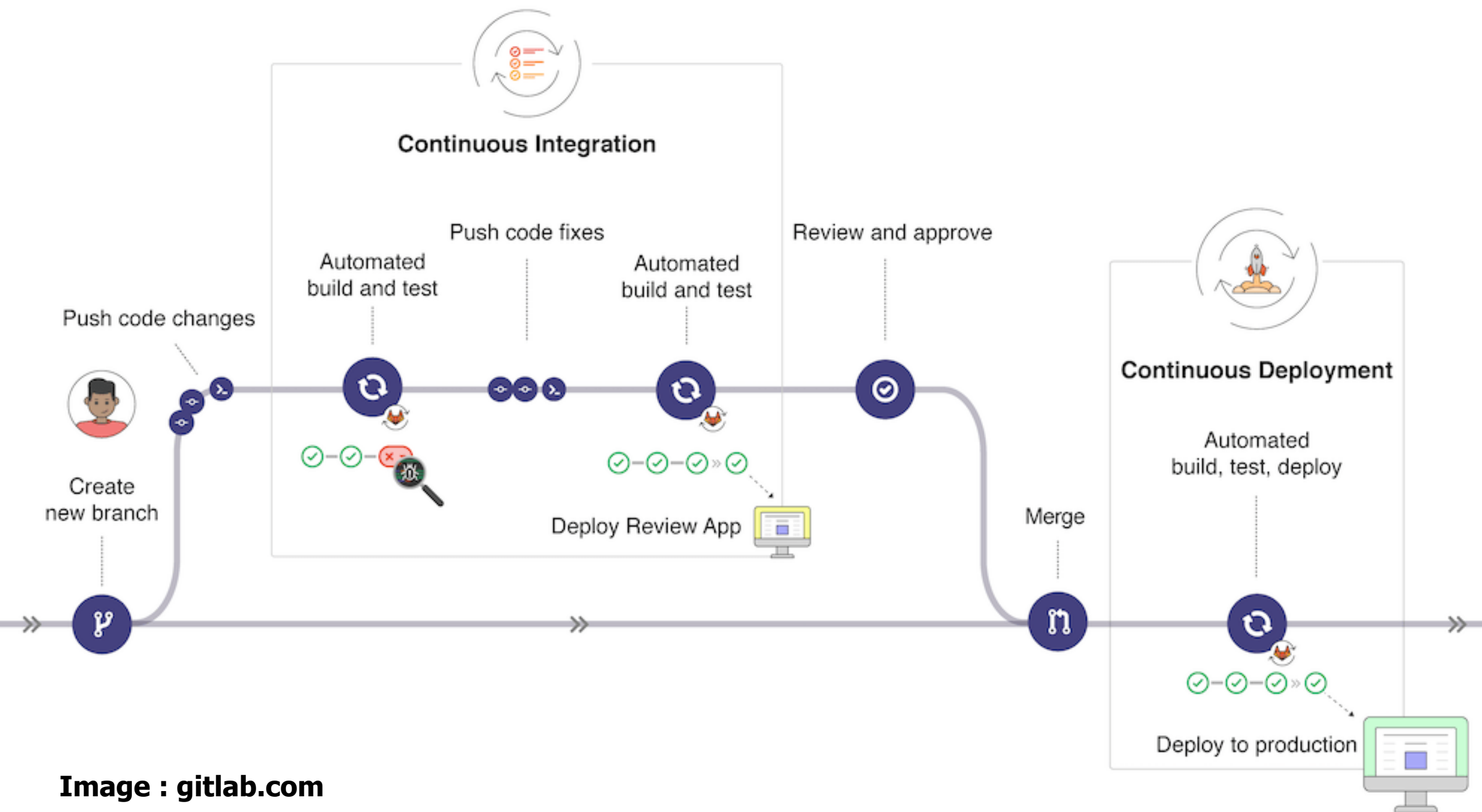
- ...

La solution : intégration continue

WORKFLOW

1. Faire des modifications **en local**
2. Faire passer **TOUS** les tests **en local**
3. Fusionner les modifications sur le serveur (git push)
4. Exécuter **TOUS** les tests sur le serveur (CI/CD)
5. Déployer le produit final et retester (CI/CD)

CI/CD est automatique (à chaque push)



CI/CD et GitLab

Set up CI/CD

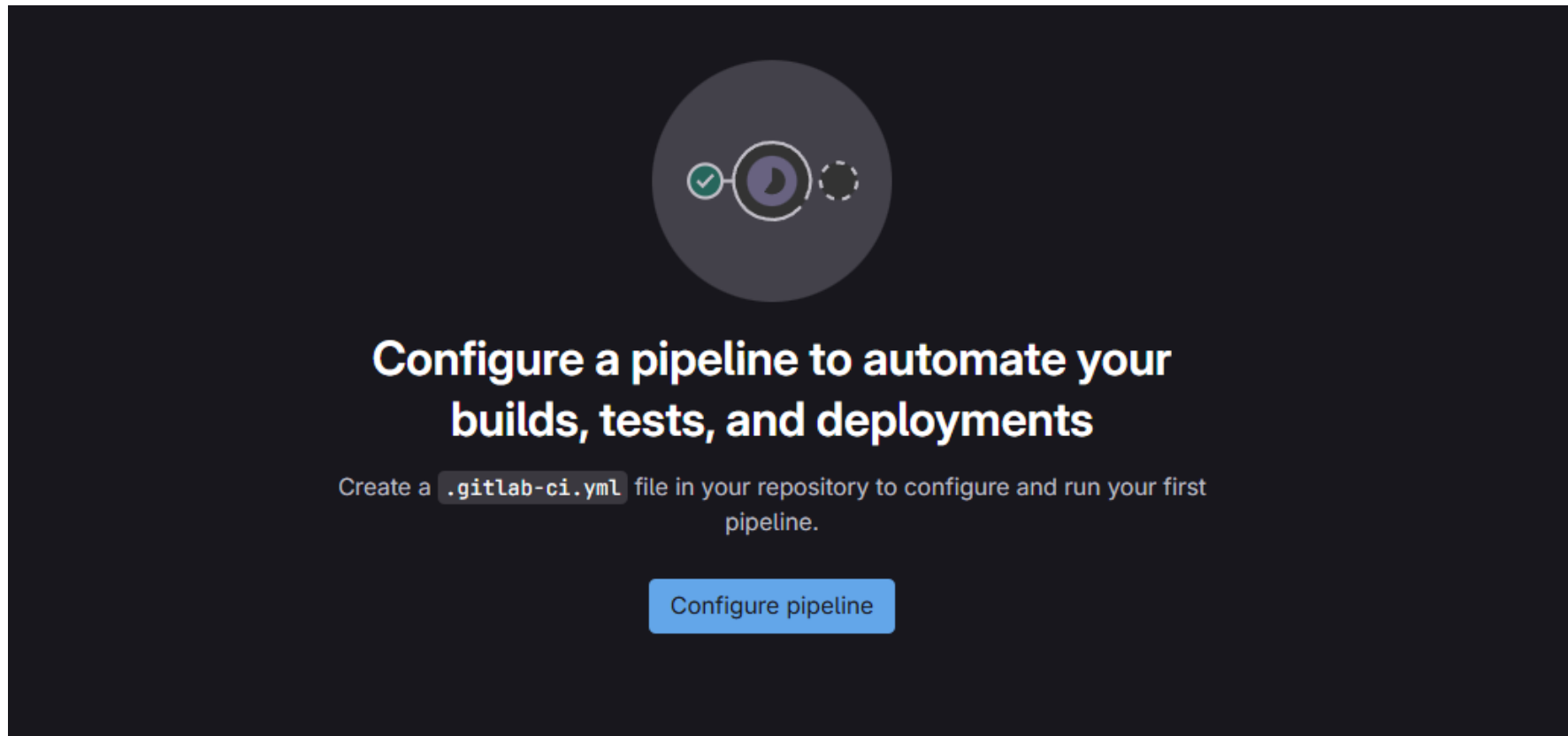
The screenshot displays the GitLab web interface for a new project named '2025-2026-testcicd'. The interface is dark-themed. At the top, there are tabs for 'main' and '2025-2026-testcicd', along with buttons for 'Find file' and 'Code'. Below this, a section titled 'Initial commit' shows that 'Le Callennec Benoit' authored the commit 32 seconds ago, with the commit hash 'a928af16' and a 'History' link. A table lists the files in the commit:

Name	Last commit	Last update
README.md	Initial commit	33 seconds ago

Below the table, the 'README.md' file is shown, containing the project name '2025-2026-testcicd' and sections for 'Getting started' and 'Add your files'. The 'Getting started' section includes instructions on how to get started with GitLab. The 'Add your files' section has two options: 'Create or upload files' and 'Add files using the command line or push an existing Git repository with the following command:'. On the right side, the 'Project information' sidebar shows statistics: 1 Commit, 1 Branch, 0 Tags, and 4 KiB Project Storage. Below these are links to 'README', 'Add LICENSE', 'Add CHANGELOG', 'Add CONTRIBUTING', 'Enable Auto DevOps', 'Add Kubernetes cluster', 'Set up CI/CD' (which is circled in red), 'Add Wiki', and 'Configure Integrations'. At the bottom of the sidebar, it says 'Created on November 10, 2025'.

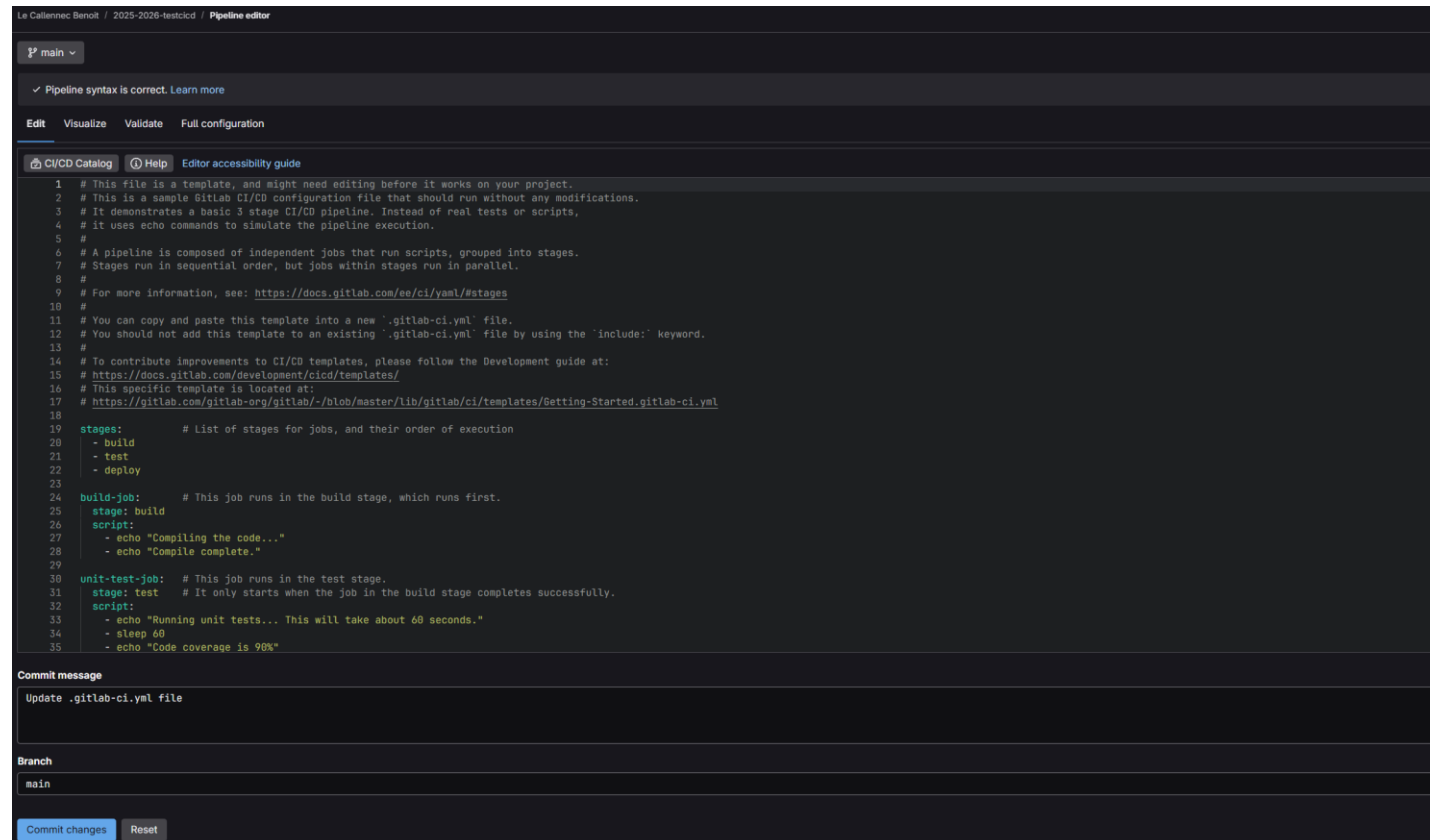
CI/CD et GitLab

Set up CI/CD



CI/CD et GitLab

Set up CI/CD



```
1 # This file is a template, and might need editing before it works on your project.
2 # This is a sample GitLab CI/CD configuration file that should run without any modifications.
3 # It demonstrates a basic 3 stage CI/CD pipeline. Instead of real tests or scripts,
4 # it uses echo commands to simulate the pipeline execution.
5 #
6 # A pipeline is composed of independent jobs that run scripts, grouped into stages.
7 # Stages run in sequential order, but jobs within stages run in parallel.
8 #
9 # For more information, see: https://docs.gitlab.com/ee/ci/yaml/#stages
10 #
11 # You can copy and paste this template into a new '.gitlab-ci.yml' file.
12 # You should not add this template to an existing '.gitlab-ci.yml' file by using the 'include:' keyword.
13 #
14 # To contribute improvements to CI/CD templates, please follow the Development guide at:
15 # https://docs.gitlab.com/development/cicd/templates/
16 # This specific template is located at:
17 # https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Getting-Started.gitlab-ci.yml
18
19 stages:           # List of stages for jobs, and their order of execution
20 - build
21 - test
22 - deploy
23
24 build-job:        # This job runs in the build stage, which runs first.
25   stage: build
26   script:
27     - echo "Compiling the code..."
28     - echo "Compile complete."
29
30 unit-test-job:    # This job runs in the test stage.
31   stage: test     # It only starts when the job in the build stage completes successfully.
32   script:
33     - echo "Running unit tests... This will take about 60 seconds."
34     - sleep 60
35     - echo "Code coverage is 98%"
```

Commit message

Update .gitlab-ci.yml file

Branch

main

Commit changes Reset

CI/CD et GitLab

Choisir l'image de base : `image: gcc`

```
build-job
✓ Passed Started 45 seconds ago by Le Callennec Benoit

Search visible log output

1 Running with gitlab-runner 18.5.0 (bda84871)
2   on alpine docker new QNVfc63uZ, system ID: s_85021cea14e6
3 Resolving secrets
4 Preparing the "docker" executor 00:01
5 Using Docker executor with image gcc ...
6 Using effective pull policy of [if-not-present] for container gcc
7 Using locally found image version due to "if-not-present" pull policy
8 Using docker image sha256:3f9e3f6a67ddbc67f32389fb2e70417f2b760178c8706c64c32c1dd47e975429 for gcc with digest gcc@sha256:25c14aaf8b2366a752f5c88ec2ff2a9ac6a6ef969089803e07c26a479be7b070 ...
9 Preparing environment 00:01
10 Using effective pull policy of [if-not-present] for container sha256:ebd69e732c981ad41cae6d485f951cc2e9c19462f27da35d25b607632e89ced1
11 Running on runner-qnvfc63uz-project-5475-concurrent-0 via srvz-ing-labinfo-worker...
12 Getting source from Git repository 00:01
13 Git! correlation ID: 01K9PQ9M2M961VV62NGF12RXNV
14 Fetching changes with git depth set to 20...
15 Initialized empty Git repository in /builds/benoit.lecallen/2025-2026-testciocd/.git/
16 Created fresh repository.
17 Checking out ea478ccf as detached HEAD (ref is main)...
18 Skipping Git submodules setup
19 Executing "step_script" stage of the job script 00:00
20 Using effective pull policy of [if-not-present] for container gcc
21 Using docker image sha256:3f9e3f6a67ddbc67f32389fb2e70417f2b760178c8706c64c32c1dd47e975429 for gcc with digest gcc@sha256:25c14aaf8b2366a752f5c88ec2ff2a9ac6a6ef969089803e07c26a479be7b070 ...
22 $ echo "Compiling the code..."
23 Compiling the code...
24 $ echo "Compile complete."
25 Compile complete.
26 Cleaning up project directory and file based variables 00:00
27 Job succeeded
```

CI/CD et GitLab

Définir les **stages** (défaut : `.pre`, `build`, `test`, `deploy`, `.post`)

Les **stages** contrôlent l'ordre d'exécution des **jobs** :

Même **stage** → les **jobs** tournent en parallèle

Sinon → les **jobs** attendent les **jobs** du **stage** précédent

CI/CD et GitLab

Installer des outils particuliers : cmake et ninja

`before_script:`

- `apt-get update --yes`
- `apt-get install --yes cmake`
- `apt-get install --yes ninja-build`

Compiler le tout

```
.gitlab-ci.yml 1.21 KiB
1  image: gcc
2
3  before_script:
4    - apt-get update --yes
5    - apt-get install --yes cmake
6    - apt-get install --yes ninja-build
7
8  stages:          # List of stages for jobs, and their order of execution
9    - build
10   - test
11   - deploy
12
13  build-job:       # This job runs in the build stage, which runs first.
14    tags:
15      - alpine-docker
16    stage: build
17    script:
18      - mkdir BUILD
19      - cd BUILD
20      - cmake -G "Ninja" ..
21      - ninja
22      - ./2243.2_Main
23
24  unit-test-job:   # This job runs in the test stage.
25    stage: test    # It only starts when the job in the build stage completes successfully.
26    script:
27      - echo "Running unit tests... This will take about 60 seconds."
28      - sleep 60
29      - echo "Code coverage is 90%"
30
31  lint-test-job:   # This job also runs in the test stage.
32    stage: test    # It can run at the same time as unit-test-job (in parallel).
33    script:
34      - echo "Linting code... This will take about 10 seconds."
35      - sleep 10
36      - echo "No lint issues found."
37
38  deploy-job:      # This job runs in the deploy stage.
39    stage: deploy  # It only runs when *both* jobs in the test stage complete successfully.
40    environment: production
41    script:
42      - echo "Deploying application..."
43      - echo "Application successfully deployed."
```

Compiler le tout

```
106 $ mkdir BUILD
107 $ cd BUILD
108 $ cmake -G "Ninja" ..
109 -- The C compiler identification is GNU 15.2.0
110 -- The CXX compiler identification is GNU 15.2.0
111 -- Detecting C compiler ABI info
112 -- Detecting C compiler ABI info - done
113 -- Check for working C compiler: /usr/bin/cc - skipped
114 -- Detecting C compile features
115 -- Detecting C compile features - done
116 -- Detecting CXX compiler ABI info
117 -- Detecting CXX compiler ABI info - done
118 -- Check for working CXX compiler: /usr/local/bin/c++ - skipped
119 -- Detecting CXX compile features
120 -- Detecting CXX compile features - done
121 -- Configuring done (0.6s)
122 -- Generating done (0.0s)
123 -- Build files have been written to: /builds/benoit.lecallen/2243.2-gelo1-benoit-le-callennec/BUILD
124 $ ninja
125 [1/8] Building CXX object Backend/CMakeFiles/2243.2_Backend.dir/backend.cpp.o
126 [2/8] Building CXX object CMakeFiles/2243.2_Main.dir/main.cpp.o
127 [3/8] Building CXX object Middleware/CMakeFiles/2243.2_Middleware.dir/middleware.cpp.o
128 [4/8] Linking CXX static library Backend/lib2243.2_Backend.a
129 [5/8] Linking CXX static library Middleware/lib2243.2_Middleware.a
130 [6/8] Building CXX object GUI/CMakeFiles/2243.2_GUI.dir/gui.cpp.o
131 [7/8] Linking CXX static library GUI/lib2243.2_GUI.a
132 [8/8] Linking CXX executable 2243.2_Main
133 $ ./2243.2_Main
134 GUI Middleware Backend
135 Cleaning up project directory and file based variables
136 Job succeeded
```

EXERCICE

OBJECTIF

Configurer + intégrer Hello_Cmake dans le CICD